นายปัณณวิชญ์ สิทธิตัน sec1 No.682110181

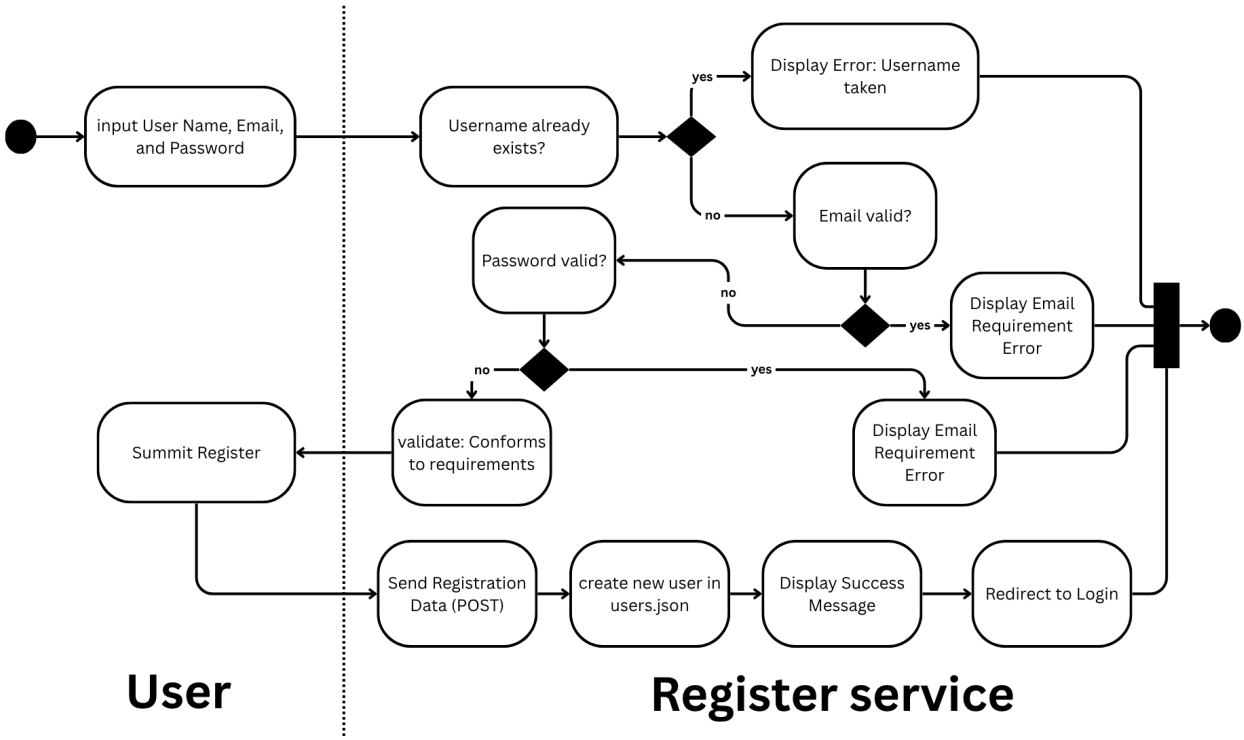
Implement your register feature, follow this Logic:

1. Request the name of the user, email for username, and password with at least 8 characters, with at least one special
2. character (like !, @, #, $, %, ^, &, *) and one uppercase character.
3. Check the password conditions on the frontend.
4. Check for existing username on the backend using users.json.
5. Store new register uses in users.json.

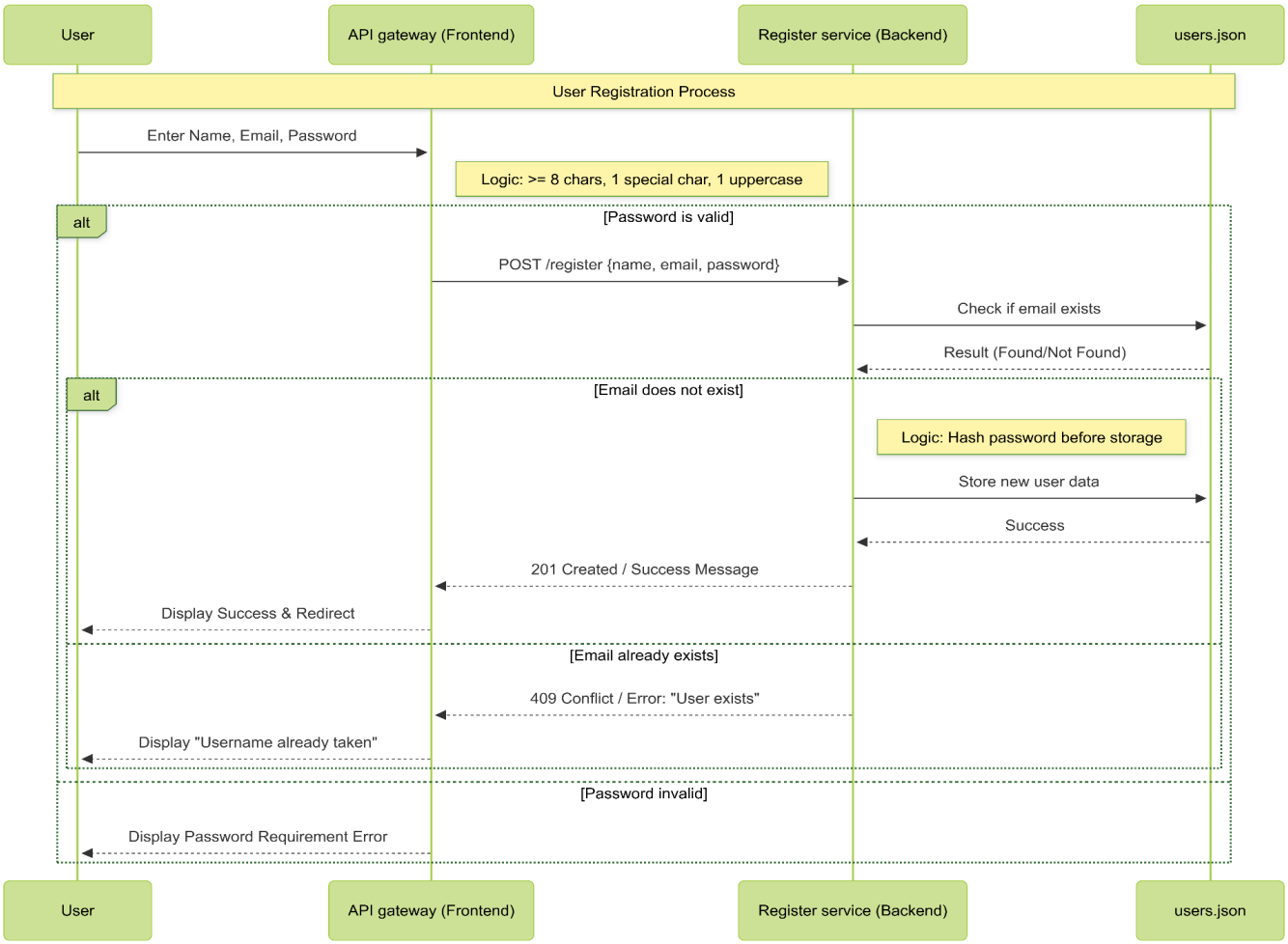
Contract Table

Component	Request (The Order)	Response (The Delivery)
Method	POST	-
Endpoint (URL)	/api/register	-
Headers	Accept: application/json	Content-Type: application/json
Status Code	-	201 Created (Success) or 400/409 (Error)
Body (Data)		[{ "username": "Alice", "email": "alice.w@example.com", "password_hash": "dac7accc1bdf78b0b3937bf67b1be2c2", "registration_date": "2024-01-15T08:22:10Z" }]

Activity Diagram



Sequence Diagram



Write GenAI Prompt using the "Contract" table + Diagrams.

Generate the backend implementation for a user registration feature using Express.js and Node.js

Express.js Route Requirement:

- **Method:** POST
- **Endpoint:** `/api/register`
- **Input Body:** The request body will contain `name`, `email` (to be used as the username), and `password`

Registration Logic (Backend):

1. Process the incoming registration request data (`name`, `email`, `password`).
2. Check for an existing username (email) on the backend by searching the `users.json` file.
3. If the username is found, return a `409 Conflict` status code.
4. If the username is new, securely hash the password. (The frontend is assumed to have already checked password conditions: min 8 characters, 1 special character, 1 uppercase character).
5. Store the new user details, including `username`, `email`, `password_hash`, and a generated `registration_date`, into the `users.json` file.

Response Requirements:

- **Success:** Return status code `201 Created`.
- **Error:** Return status code `400` (Bad Request) for validation issues or `409` (Conflict) if the username already exists.
- **Headers:** `Content-Type: application/json`.

Code Structure:

- Create an Express.js route specifically for the `POST /api/register` request.
- Implement the core registration logic (username check and user storage) in a separate file named `register.js`.
- Ensure all code includes comments explaining the functionality and logic flow.